#Polymorphism

Question 1

Create a class Calculator that can perform addition in multiple ways:

If two numbers are passed → return their sum
If three numbers are passed → return their sum
If a list of numbers is passed → return the sum of all elements

---

Question 2

Create a class Vector to represent a 2D vector with attributes x and y.

Overload:

the + operator to add two vectors
the * operator to compute the dot product of two vectors

---

3. Ride-Hailing App – Fare Calculation
Scenario

You are building a system similar to Uber/Ola where different ride types calculate fare differently.

Requirements

Create:

Base class: Ride
Subclasses: BikeRide, CarRide, AutoRide

Each class should implement:

def calculate_fare(self, distance):
Task
BikeRide: ₹5 per km
CarRide: ₹10 per km + ₹20 base fare
AutoRide: ₹8 per km

Write code that:

Stores all ride objects in a list
Calls calculate_fare() polymorphically in a loop

4.Payment Gateway System – Method Overriding
Scenario

An e-commerce platform supports multiple payment methods.

Requirements

Create:

Base class: Payment
Subclasses:
CreditCardPayment
UPIPayment
WalletPayment

Each class overrides:

def process_payment(self, amount):
Task

Simulate processing ₹1000 using each payment type and print different processing messages.

Concepts tested

Method overriding
Same interface, different behavior
Employee Performance System – Polymorphic Bonus Calculation Scenario

In a company, bonus calculation varies based on role.

Requirements

Create:

Employee (base class)
Subclasses:
Developer
Manager
Intern

Each class implements:

def calculate_bonus(self, salary):
Rules
Developer → 10% bonus
Manager → 20% bonus
Intern → fixed ₹2000 bonus
Task

Create multiple employee objects and compute bonuses using a loop.

This simulates HR/payroll systems, a common interview scenario.

5.File Export System – Polymorphism with Common Interface Scenario

You are designing a system that exports reports in multiple formats.

Requirements

Create:

Base class: ReportExporter
Subclasses:
PDFExporter
ExcelExporter
CSVExporter

Each class overrides:

def export(self, data):
Task

Write code that:

Accepts a list of exporter objects
Calls export() on each

This tests understanding of plug-and-play architecture using polymorphism, commonly asked in design interviews.
---

6. Operator Overloading – Real-Time Banking Transactions
Scenario

You are developing a banking system where accounts can be merged.

Requirements

Create class:

class BankAccount:

def __init__(self, balance):

Overload:

def __add__(self, other):
Behavior

Adding two accounts should return a new account with combined balance.

Example
a1 = BankAccount(5000)
a2 = BankAccount(3000)

a3 = a1 + a2
print(a3.balance)   # 8000

This question tests:

Operator overloading
Object creation inside special methods
---

7. Notification System – Polymorphism with Abstract-like Behavior
Scenario

A system sends notifications via different channels.

Requirements

Create:

Notification base class with method:
def send(self, message):

Subclasses:

EmailNotification
SMSNotification
PushNotification

Each should implement its own sending logic.

Task

Write a function:

def notify_user(notification_obj, message)